РЕКЛАМА · YANDEX.RU
Опрос про онлайн-образование
Поучаствуйте в исследовании
Реклама

filatovvv 21 фев в 11:00

Делаю навигацию в приложении на Compose

Средний 12 мин 1K

Разработка мобильных приложений*, Разработка под Android*, Kotlin*, Jetpack Compose*

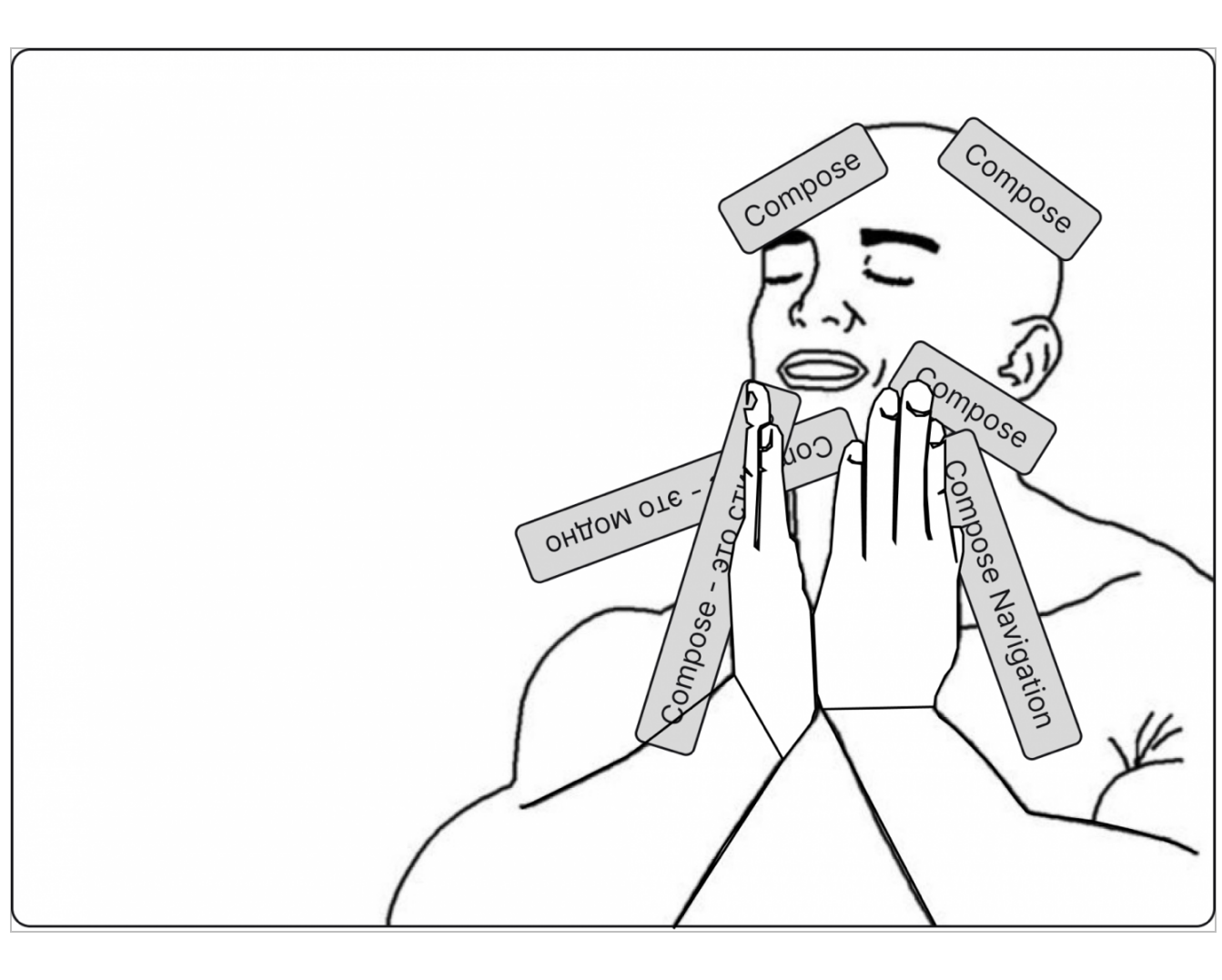
Кейс

Это межмодульная навигация, салаги
Routers Binders Navigator NavCommands
Хм.. На еду не похоже
Витек - Dev
Витек, че это?
FILL KEYS
NAVIGATION

О чем ты нам расскажешь и кто ты такой?

То, о чем пойдет речь ниже, назвать инновацией нельзя. Это, скорее, мое личное видение по навигации между фичевыми модулями, которое я определил для своего проекта базирующегося на Compose и Compose Navigation.

В моей предыдущей **статье** я писал, что проект в рамках которого я рассматриваю разные подходы — делается на всем, что связано с Compose и разрабатывается так, чтобы каждую его запчасть можно было максимально переиспользовать на других проектах и, в перспективе, доработать проект под кросс платформу на KMP.



Ну ок, а кто же будет вещать?

Меня зовут Виктор и я мобильный разработчик под Android с опытом в этой сфере 7+ лет и по совместительству основатель небольшой команды мобильной разработки Fill Team. Успел поработать как в маленьких стартапах, так и в крупных российских и зарубежных компаниях. Тем не менее на лютую экспертность не претендую, а лишь хочу поделиться своим опытом о том как я навигацию в своем приложении делал.

Для ориентира по статье

- Фичи нужно переиспользовать
- А было ли что-то подходящее?
- К делу
- Какой функционал хочется? Чего делаем?
- Сформулируем задачу предметно
- Шагаем необходимые шаги
- Про настройку навигации внутри фичи
- Про интеграцию фичи в проектный флоу
- Ну ок, сделали, а как применять?
- Здесь наши полномочия все

Для ориентира по серии статей

- Как я приложение на Compose писал
- Делаю навигацию в приложении на Compose
- скоро больше…

Фичи нужно переиспользовать

Так уж повелось, что если я что-то делаю, то всегда пытаюсь сделать так, чтобы результат работы можно было использовать в смежных областях (*такой подход позволяет, количественно перекрывать многие потребности в будущем*).

Аналогичного принципа я придерживался, когда начинал писать приложение для хранения паролей на Compose.

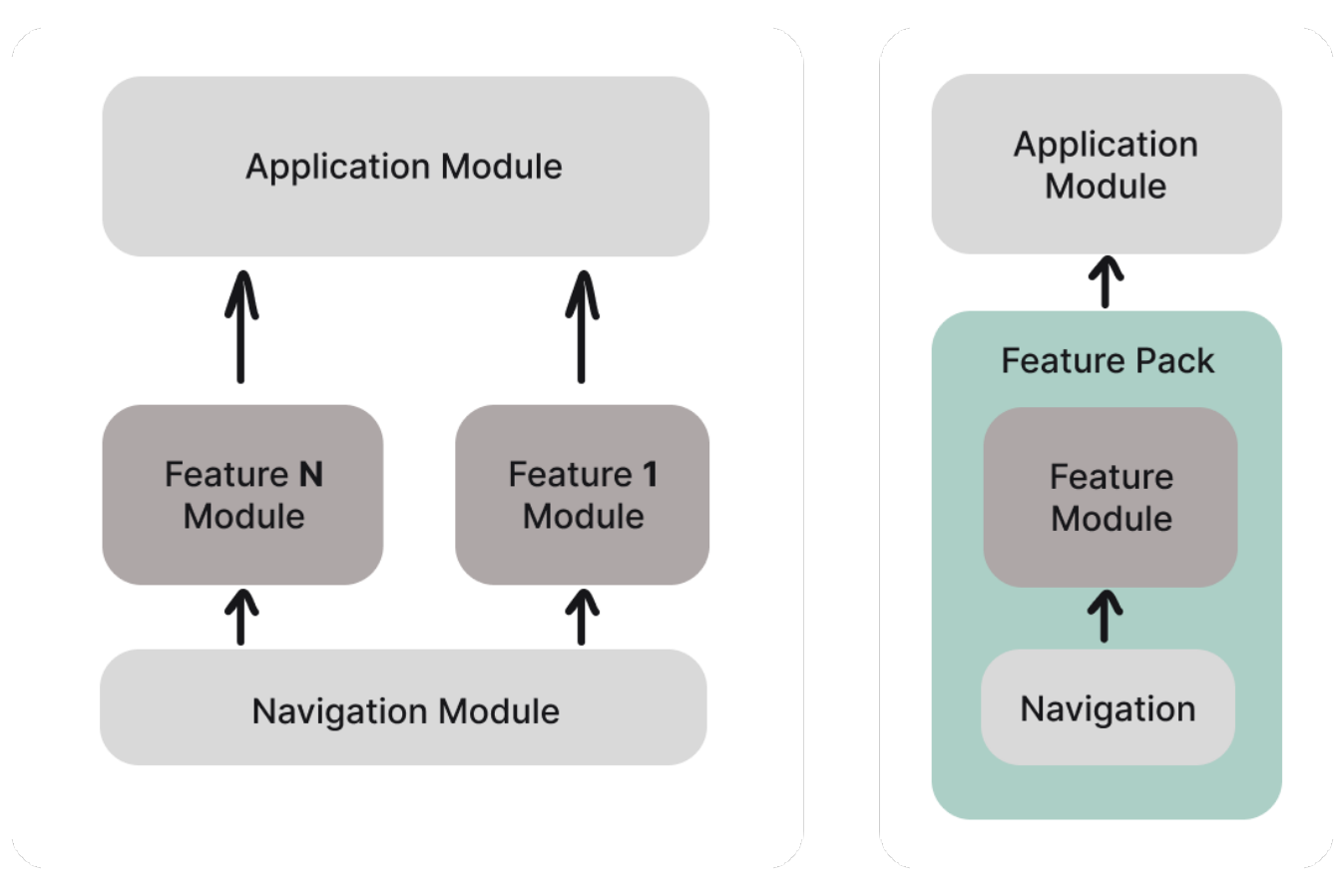
> *Дело в том, что в рамках моей команды Fill Team к нам часто приходят заказчики у которых сравнительно небольшой бюджет, но при этом эти клиенты, которым нужно MVP для проверки гипотезы. А MVP, как мы знаем, очень часто состоит из нескольких типовых фичей: аутентификация; регистрация; настройки и т.д.. Делать проект абсолютно с нуля: дорогая и невыгодная затея, а вот решение с повторным использованием готовых фичей — поистине оптимальное для таких ситуаций. Но не будем закапываться в бизнесовые детали, статья не об этом…*

В общем, когда проектировал архитектуру приложения для хранения паролей, стояла задача переиспользования фич как полноценные модули из которых потом можно будет собирать новые приложения и при этом затрачивать минимум времени. Соответственно, навигация внутри фичей должна быть закрыта и при этом фичи не должны знать ничего о других фичах (*а значит и навигация напрямую из одной фичи в другую должна подниматься на уровень выше этих N фичей*).

А было ли что-то подходящее?

Как я писал выше, для Compose Navigation, на момент проектирования архитектуры приложения, подходящих решений под мои требования я не нашел.

Хотелось сделать удобное в использовании и не перегруженное лишними деталями решение, которое в перспективе можно будет расширять. Скажем, сделать модуль с перечнем навигационных контрактов, который можно будет подключать к модулям фичей и дружить с Compose Navigation. В итоге на ум пришла следующая схема подключения (*довольно типовая, на самом деле*):

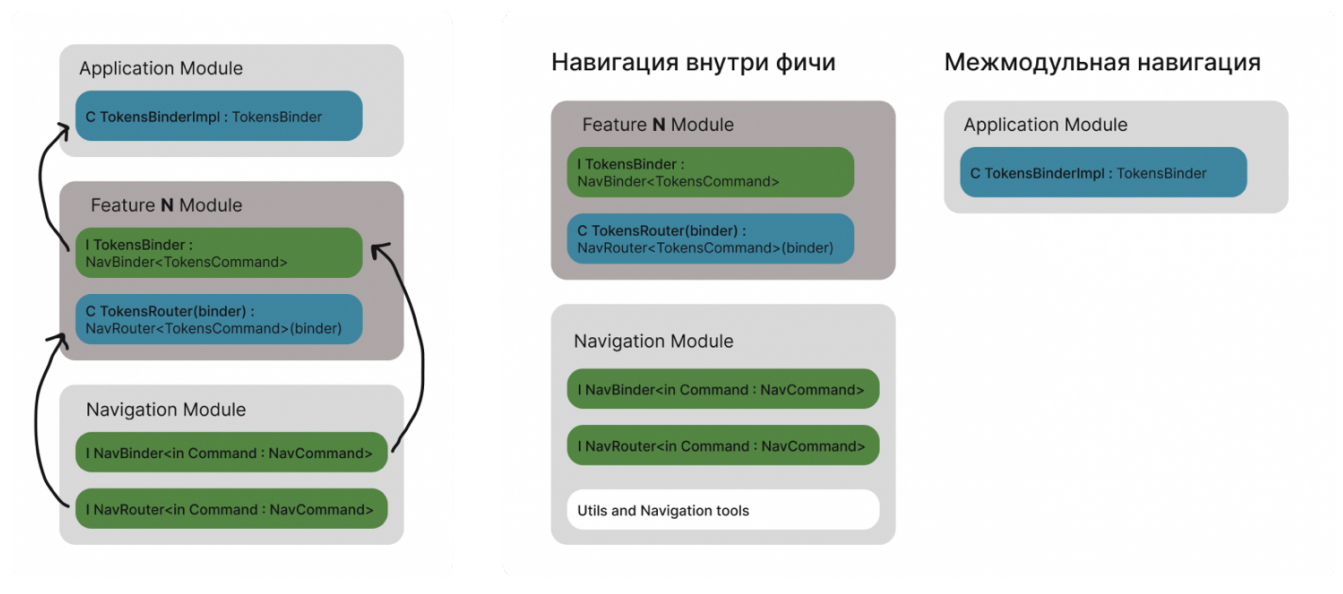


В дальнейшем нам будут интересны Navigation Module и Feature Pack. Первый — это как раз таки модуль содержащий все необходимые контракты и базовые классы навигации. Второй — это абстракция над инкапсулированной логикой фичи (*помним, у нас фича — это максимально замкнутая система со своей внутренней навигацией*).

К делу

На этапе проектирования решил, что модуль определяющий контракты и базовые реализации связанные с навигацией должен быть отдельным, содержать все необходимое, что так или иначе связано с настройками навигации и переходов. При этом модуль, в перспективе, должен быть собран в отдельный подключаемый файл, который можно будет просто подключать к фичам в gradle.

Постановка сделана, го к решению. Пришлось набросать несколько блок схем, которые графически отобразили бы поток мыслей в связанную систему. Получилось что-то такое:


Application Module
C TokensBinderImpl : TokensBinder
Feature N Module
I TokensBinder : NavBinder<TokensCommand>
C TokensRouter(binder) : NavRouter<TokensCommand>(binder)
Navigation Module
I NavBinder<in Command : NavCommand>
I NavRouter<in Command : NavCommand>
Навигация внутри фичи
Feature N Module
I TokensBinder : NavBinder<TokensCommand>
C TokensRouter(binder) : NavRouter<TokensCommand>(binder)
Navigation Module
I NavBinder<in Command : NavCommand>
I NavRouter<in Command : NavCommand>
Utils and Navigation tools
Межмодульная навигация
Application Module
C TokensBinderImpl : TokensBinder

Navigation Module определяет парочку логических блоков:

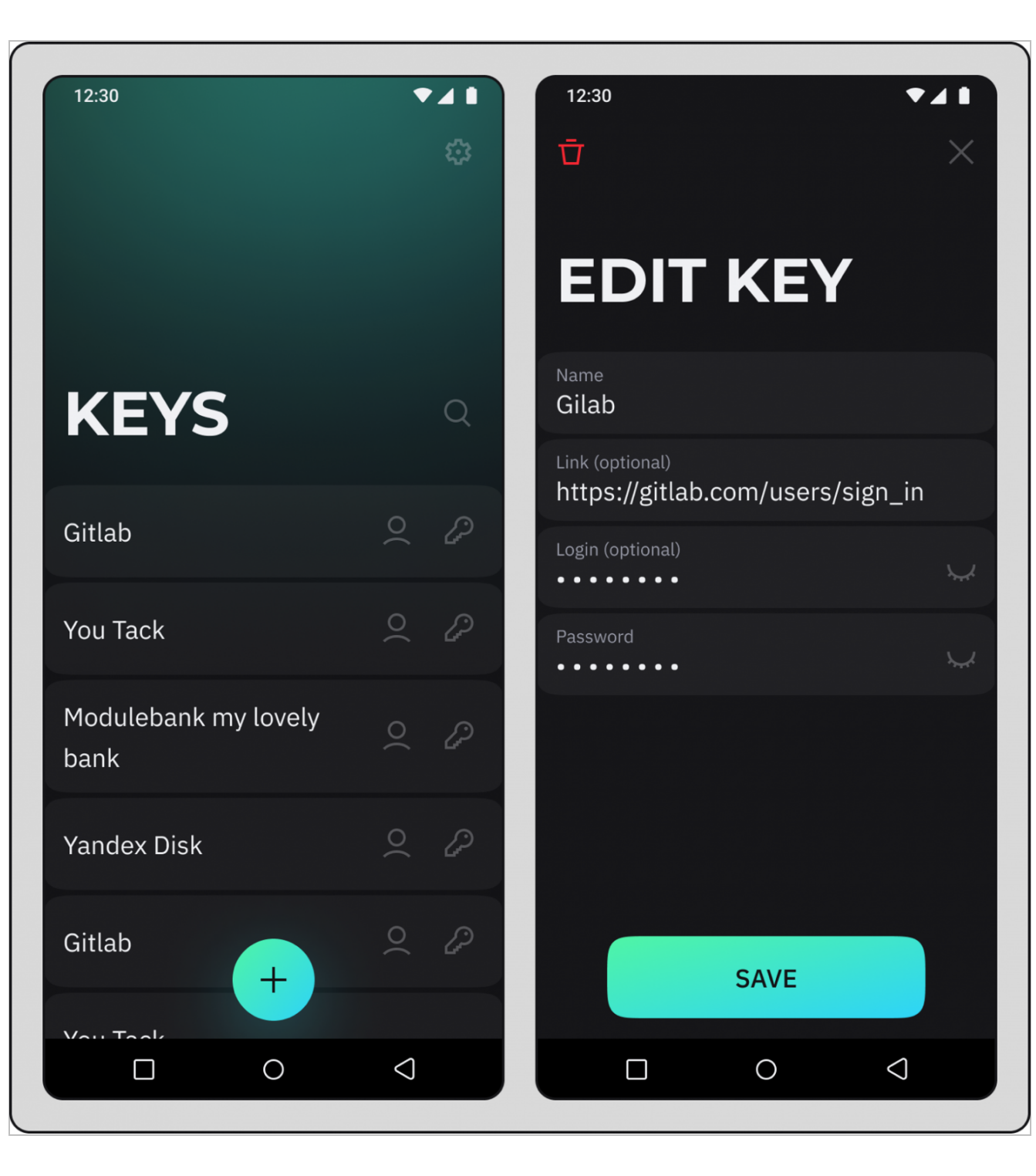
- Интерфейсы и классы, отвечающие за взаимодействие с навигацией как внутри фичи так и за ее пределами (*на рисунке показаны основные интерфейсы в Navigation Module*)
- Упрощающие жизнь надстройки, такие как: системный backpressure, backpressure с условием, таймером, анимации и т.д.. (*см. Utils and Navigation tools*)

> Ничто так хорошо не расскажет о решении, как его применение на практике!

Подумал я и решил рассказать на примере одной фичи из проекта, как это все работает.

Какой функционал хочется? Чего делаем?

А делаем фичу для работы с паролями состоящую из двух экранов: отображение списка паролей; создание и редактирование пароля. В приложении они выглядят как на рисунке, соответственно.



Далее, вместо слова «пароль», часто буду использовать термин «токен», так как в будущем приложение будет работать не только с паролями, но и с другими данными, такими как: карты; pdf документы; текст; и т.д..

Сформулируем задачу предметно

Необходимо реализовать навигацию для модуля фичи токенов. Навигация между экранами фичи должна быть инкапсулирована и неизменна за пределами фичи. Навигация модуля должна давать ручки для входа и выхода из фичи (*то есть, дать возможность встраивать ее в любые другие флоу имея лишь информацию о том, как фича может завершаться и инициализироваться*).

Шагаем необходимые шаги

Весь процесс реализации навигации для модуля можно разделить на два последовательных этапа, каждый из которых состоит из нескольких шагов:

1. **Настройка навигации на этапе реализации фичи**
    1. Реализация команд переходов (*каждая команда определяет возможный переход между экранами или же переход соответствующий завершению фичи*)
    2. Подготовка данных, участвующих в инициализации экранов фичи
    3. Реализация графа навигации (*в нашем случае, на базе Compose Navigation*)
    4. Определение интерфейса обработчика терминальных состояний фичи
    5. Реализация роутера для внутрифичевой навигации
2. **Интеграция фичи в проектный флоу**
    1. Реализация интерфейса обработчика терминальных состояний фичи

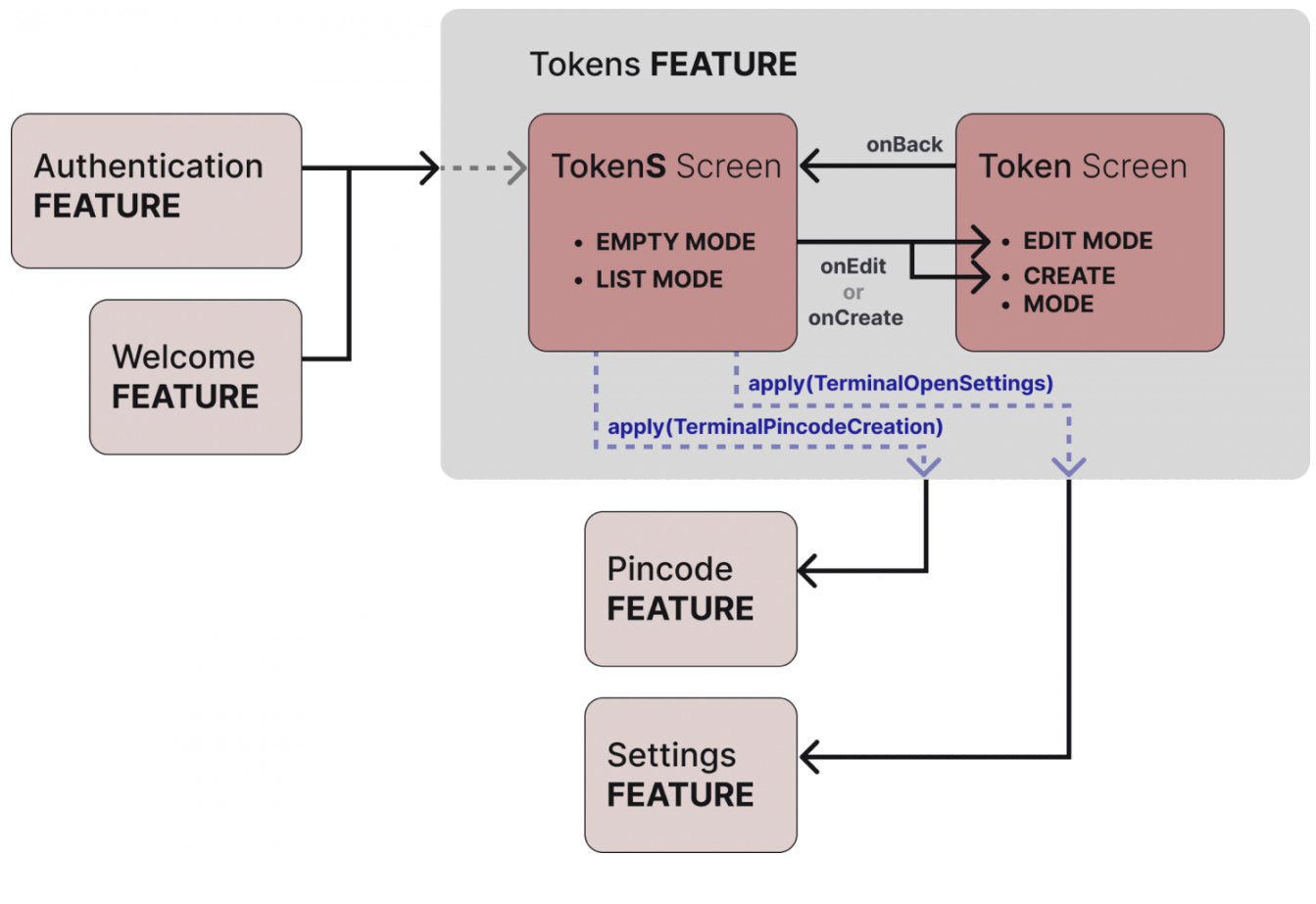
Как можно заметить выше в иерархии, самый объемный этап — первый. Но важно понимать, что он выполняется только в процессе реализации самой фичи и в дальнейшем при использовании фичи в различных проектах уже не изменяется, а становится закрыт за занавеской инкапсуляции.

Про настройку навигации внутри фичи

Реализация команд переходов

На этапе проектирования (*или даже на этапе прототипирования схемы экранов*) мы первым делом можем составить блок-схему всех переходов фичи. Такой подход позволит нам сразу понять как будут взаимодействовать экраны фичи между собой и как фича будет взаимодействовать с остальными флоу приложения.

Пример схемы переходов для фичи токенов выглядит следующим образом.



На базе схемы полученной на этапе проектирования сразу видно какие переходы будут внутренними, а какие будут смотреть наружу для работы с ними на этапе интеграции фичи в проектный флоу. Имея такую информацию, мы можем сразу определить изолированную иерархию команд, которые будут использоваться для обработки переходов в коде приложения.

‣ Иерархия команд выглядит таким образом

Те команды, которые помечены внутренними, реализуются только на этапе создания модуля фичи. Команды же, которые помечены терминальными, используются на этапе интеграции фичи в проектный флоу в соответствующий Binder (*о нем пойдет речь дальше*).

Инициализирующие данные

Мы знаем, что фича может состоять из нескольких отдельных экранов, каждый из которых реализует свою логику работы, но может быть завязан на соседях в рамках одной фичи. В нашем случае у нас два экрана (*каждый из которых имеет несколько состояний визуализации*): экран отображения списка токенов и экран для создания/редактирования токена.

На этапе перехода на экран нам важно иметь возможность дать ему понять, в каком из состояний нужно показываться. Разным состояниям могут понадобиться дополнительные данные, которые могут участвовать в первичной инициализации. Для этого я решил использовать, так называемый объект ноды для каждого отдельного экрана.

Нода — это логическое представление как состояния инициализации экрана, так и его навигационного состояния. В случае с фичей токенов у меня есть две ноды.

‣ Нода для списка токенов (заголовок блока кода)

‣ Нода для создания/редактирования токена (заголовок блока кода)

Как вы могли заметить, каждая нода реализует два контракта: NavRouteNode.Route и NavRouteNode.Parameters.

Первый определяет интерфейс для работы с нодой на уровне графа навигации. Второй же определяет интерфейс для работы с нодой на уровне логики навигации: конфигурирование ноды; данных ноды; и т.д..

У NavRouteNode.Route есть три основных поля, которые используются на этапе реализации ноды для того, чтобы можно было взаимодействовать с навигацией уровня Compose Navigation:

- **val tag: String** — поле, которое хранит строковое представление ноды, как правило эквивалентно названию ноды/экрана (*используется как часть route и routeWithValues в отдельных случаях для построения линки перехода на экран*)
- **val route: String** — сырая линка, характеризующая экран. По ней Compose Navigation сможет понять, какой экран в графе навигации нам надо будет отрисовать (*в линке помимо основной части хранятся заглушки данных, которые могут быть нужны экрану в формате ключ-значение*)
- **val arguments: List<NamedNavArgument>** — список всех полей, которые могут быть в route. Они используются для того, чтобы Compose Navigation смог достать эти значения на этапе отрисовки экрана

В самом простом случае, когда нам нужно просто показать, что экрану не нужны никакие значения и он может быть только в одном единственном состоянии, нода экрана будет выглядеть так, как она выглядит для экрана списка токенов.

С нодами закончили, дальше у нас идет этап оформления графа навигации Compose Navigation.

Граф навигации

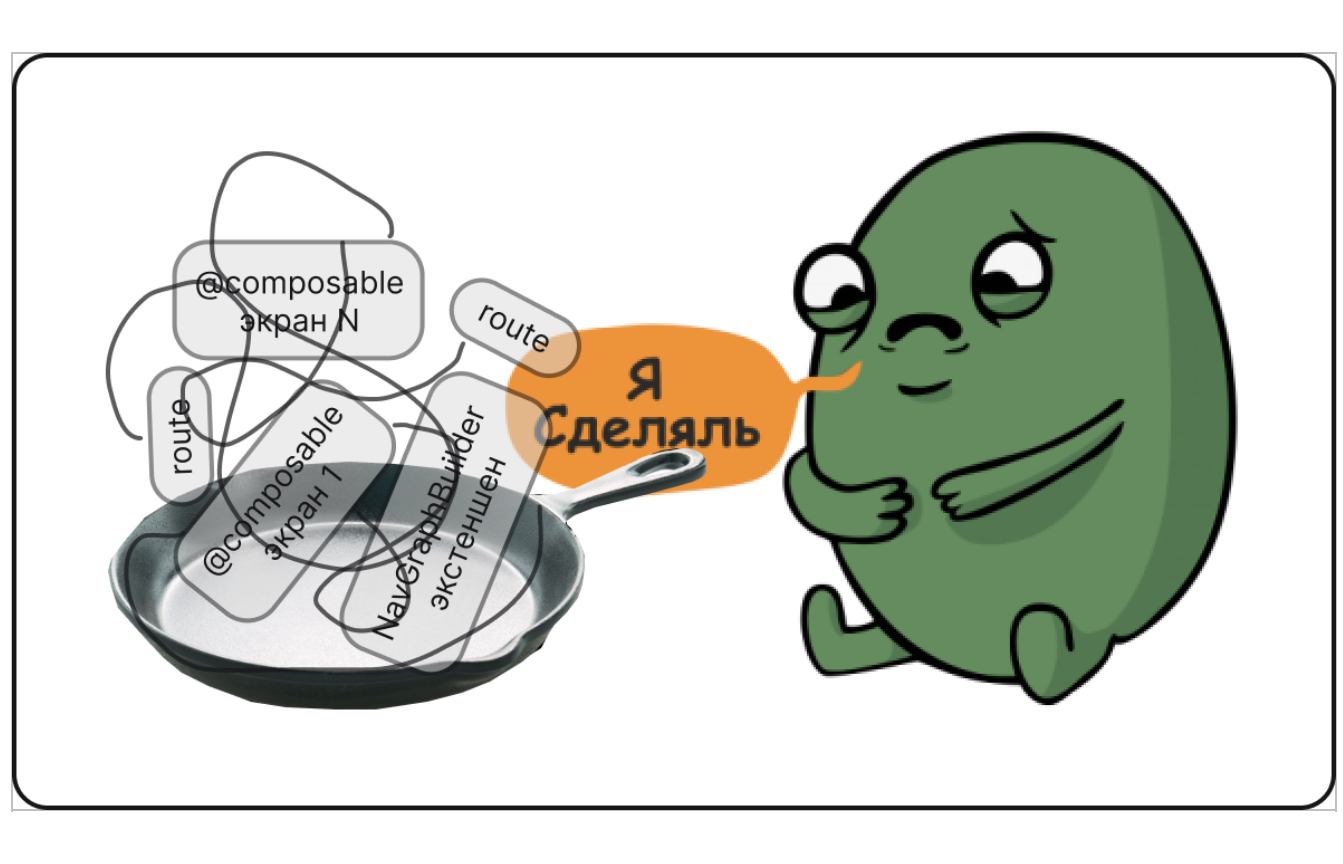
Итак, на текущем этапе мы подготовили ноды для того, чтобы построить граф навигации между экранами внутри фичи токенов. Так как фича довольно небольшая, то и граф навигации будет малюсеньким.

‣ Код графа навигации фичи токенов

Что же мы видим выше? Во-первых, если мы пользуемся Compose Navigation, то т.к. фича является отдельным подключаемым модулем и у нее должен быть свой граф навигации, мы создаем экстеншен для NavGraphBuilder. Он будет получать наш внутренний роутер для управления навигацией внутри модуля фичи. Этот экстеншен в дальнейшем вызывается в родительском графе навигации.

Во-вторых, мы размечаем каждый наш экран (*в том числе каждое отдельное состояние экрана*) с помощью composable, которая реализуется Compose Navigation для того, чтобы разметить отдельные экраны, которые в дальнейшем будут находиться при проходе по графу навигации для отрисовки.

Как мы видим, route поля нод используются, чтобы разметить экраны. А там, где на вход идут дополнительные поля с данными в route, мы пробрасываем arguments наших нод. Все довольно просто.



Интерфейс обработчика терминальных состояний

Тут все довольно просто. Нам нужно просто подготовить интерфейс для дальнейшей работы с терминальными состояниями на уровне приложения (*т.е тогда, когда мы будем интегрировать наш модуль фичи в любой из проектов*).

‣ Код интерфейса Binder'a

Выглядит супер просто, т.к. основной метод **flowTo** един для всех видов байндеров и определен на уровне интерфейса NavBinder. Здесь же мы указываем тип командной иерархии с которой наш байндер будет уметь работать.

Роутер внутрифичевой навигации

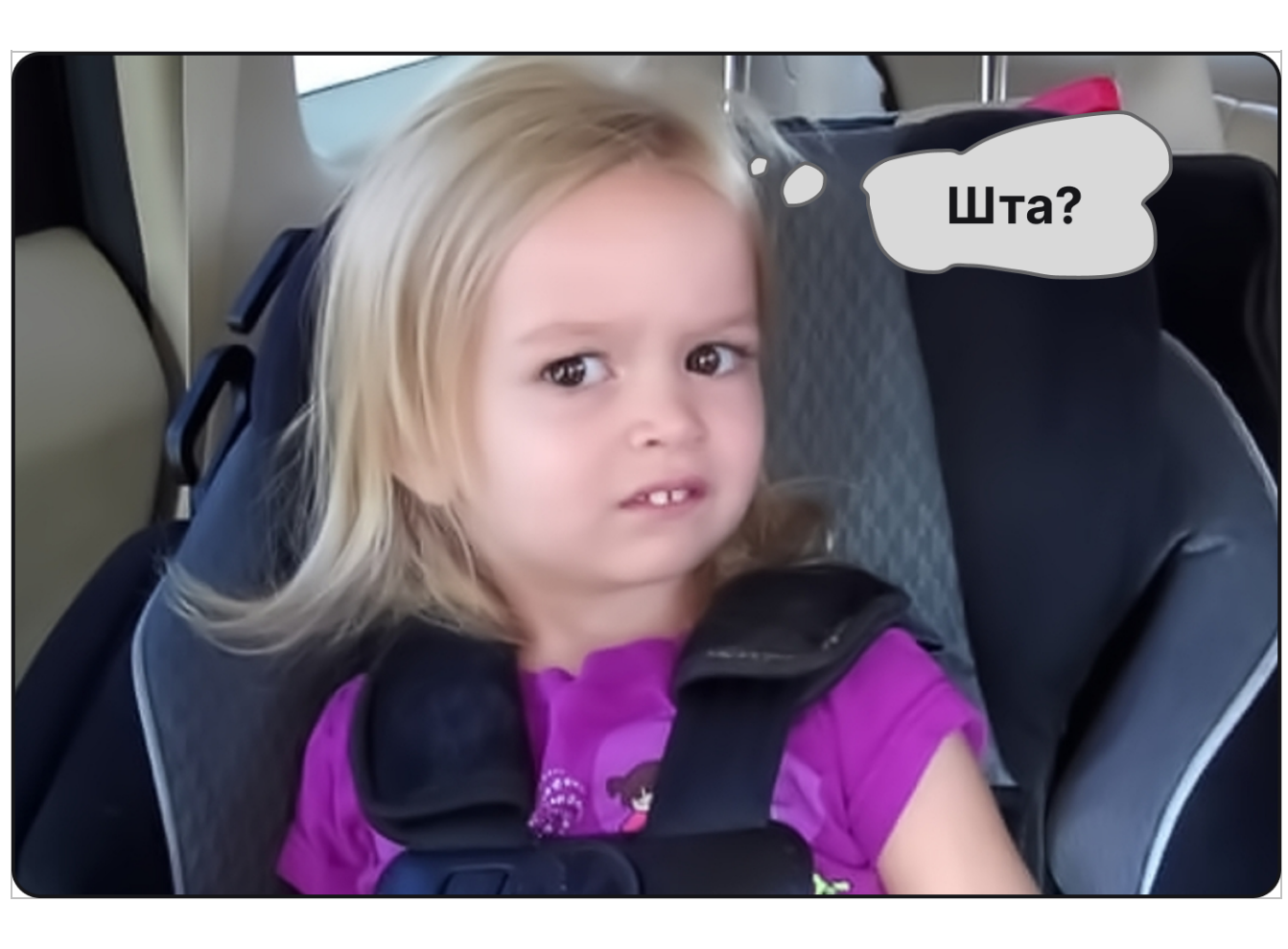
Ну и пожалуй последний муторный шаг реализации внутрифичевой навигации — это реализация роутера для обработки переходов на уровне фичи.

‣ Код роутера для навигации внутри фичи

```
class TokensRouter(override val binder: TokensBinder) : NavRouter<TokensComma
    override fun navigateTo(navController: NavController?, command: NavRouter
        if (command !is TokensCommand) {
            return false
        }

        return when (command) {
            is TokensCommand.EditToken -> {
                navController?.navigate(TokenNode.Edition(command.tokenId).ro
                true
            }
            TokensCommand.CreateToken -> {
                navController?.navigate(TokenNode.Creation.routeWithValues)
                true
            }
            else -> binder.flowTo(navController, command)
        }
    }
}
```

На самом деле, все довольно просто: если получили команду перехода на редактирование, значит нужно шагнуть с помощью navController'а на соответствующую ноду с настроенными данными состояния инициализации экрана.



На этом реализация навигации на уровне модуля фичи завершена. Самое сложное позади. Здесь у нас уже полностью готовый модуль, который мы можем упаковать. Дальше останется только взять нашу фичу, подключить ее к проекту (*в нашем случае подключить в app модуль*) и сделать последний рывок.

Про интеграцию фичи в проектный флоу

Итак, теперь остается только интегрировать нашу фичу в проектный флоу. Для этого достаточно на уровне модуля app создать реализацию определенного ранее интерфейса **TokensBinder** и описать переходы при получении соответствующих терминальных команд от нашей фичи.

В коде, наша реализация выглядит следующим образом (*подпись для блока кода*).

‣ Реализация Binder'a для в месте интеграции

Структурно реализация биндера очень похожа на роутер, но логически она полностью отделена от поведения роутера, т.к. задача роутера — это навигация внутри фичи, а задача биндера — это навигация между фичами.

Ну и так как мы ранее определили, что с фичи токенов мы можем уйти только в фичу настроек и на экран создания/изменения пинкода для входа в приложение, то и реализовывать мы будем всего два межмодульных перехода.

Вот и вся навигация на примере модуля фичи токенов. Дальше разберу несколько подробнее модуль навигации, который определяет все базовые контракты и реализации подхода навигации, который я использую в своем проекте для хранения паролей и планирую использовать в последующих.

Ну ок, сделали, а как использовать?

Да довольно просто. Если мы работаем с навигацией внутри фичи, то бросаем в конструктор TokensViewModel фичи наш Navigator и пользуемся.

‣ Пример во ViewModel

А если мы говорим про межфичевую навигацию, то для этого нам нужно лишь затолкать наш TokensGraph в основной граф приложения.

‣ Вызов в Compose Navigation корневом графе

Дальше нужно будет сделать разовую инициализацию NavigatorImpl : Navigator *(из модуля **navigation**)*, подсунуть в него наши роутеры, а в роутеры реализацию биндеров и все. В своем проекте я переложил эти обязательства на koin и выглядит это у меня примерно следующим образом:

Шаг 1. На уровне navigation модуля происходит инициализация навигатора:

```
single { NavigatorImpl(get(named(KOIN_ROUTERS_QUALIFIER))) }
```

Шаг 2. Не забываем, что нужно предоставить на уровне **app** модуля зависимость на binder's:

```
single<TokensBinder> { TokensBinderImpl() }
```

Шаг 3. Binder's будут пихаться DI в роутеры фичей по потребности, а значит не забываем про предоставление экземпляров роутеров на уровне фичей. Это делается на этапе реализации модуля фичи:

```
single { TokensRouter(binder = get()) }
```

Шаг 4. Далее на уровне app модуля я готовлю список роутеров всех фичей, что по сути является последним шагом:

```
single(named(KOIN_ROUTERS_QUALIFIER)) {
   setOf<NavRouter<*>>(
       ...,
       get<TokensRouter>(),
   )
}
```

На самом деле, шаги 1, 3, 4 реализуются в рамках проекта разово. Дальше только по мере добавления фичей, нужно будет возвращаться к шагу 2 и в шаг 4 дописывать **get<РОУТЕР_НОВОЙ_ФИЧИ>()**.

Пожалуй, это все самое основное. Если у вас остались вопросы или было что-то непонятно, пишите в комментариях — буду рад дополнить и обсудить. Так как статья итак получилась большой, то пихать сюда описание модуля navigation не буду, если читателю будет интересно, тогда просто оформлю в отдельную нано статью — так как рассказывать там немного. Может даже к моменту написания выложу модуль в публичный Gitlab репозиторий (*почему бы и да?!*).

Здесь наши полномочия все

Всем дошедшим до этих строк, огромное спасибо за ваше внимание. Надеюсь было интересно и, главное, понятно и полезно. А если заинтересовал проект, на котором я экспериментировал, то с ним можно ознакомиться в Google Play (*работает из под VPN или с телефона*).

Теги: jetpack compose, compose, приложения для android, менеджер паролей, разработка мобильных приложений, навигация в приложени, android development, senior developer, разработка под android, продвижение приложений

Хабы: Разработка мобильных приложений, Разработка под Android, Kotlin, Jetpack Compose

0 10 8 +8

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электропочта

4 Карма 0 Рейтинг

Виктор Филатов @filatovvv

Пользователь

Реклама

Комментарии 8

Paul85 22 фев в 08:48

А что с передачей parcelable аргументов?

Ответить

filatovvv 22 фев в 09:05

Тут я бы уточнил: "а действительно ли есть необходимость проталкивать объект или можно обойтись более легковесным вариантом"?

На самом деле, под большинство задач (*если не под все*) достаточно бросить пару id's, несколько примитивов, а остальное дополучить в условной ViewModel на уровне бизнес логики фичи через кейсы/репозитории/кеш/т.д..

P.S.: Но если есть необходимость передавать объекты, то можно протащить их преобразовав в строку Json (КРАЙНЕ не рекомендую так делать) и положив ее по аналогии с примитивами в route в качестве параметра.

Ответить

Paul85 23 фев в 00:52

Ну эти костыли уже попробовал)))

На самом деле когда работаешь с кривыми апи, то проще передать объект через экран, а не делать еще один запрос. Вот у нас сейчас запрос один отрабатывает 3-5 секунд, на беке это пофиксить не могут, а мне приходится его делать на одном экране, потом переходить на другой экран и еще раз это делать. Кешировать особо смысла нету, потому что при каждом входе данные меняются. Та и БД в проект не завозили, то из-за одного объекта это делать нету смысла. Получается надо хранить данные где-то в репозитории и получать их с разных экранов, что кажется еще одним костылем. Странно получить id через аргументы в навигации, а остаток полей получить из вьюмодели. В общем крайне удивлен я решением гугла сделать такое в навигации. После многих лет использования Cicerone навигация гугла выглядит очень костыльной.

Ответить

filatovvv 23 фев в 17:36

Кэширование данных в репо - как раз таки, не является костылем по причине того, что одна из ключевых задач репозитория - работа с данными, в том числе с их переиспользованием. Это вполне себе реальная практика. При этом если данные меняются после ухода с цепочки экранов, то ничто не мешает реализовать кэширование в скоупе (*например, сброс кэша при очередном попадании на экран А в цепочке А->Б*), что тоже является корректным поведением.

Но на самом деле, я бы смотрел по конкретной задаче. Очень может быть, что в вашем случае тащить кэширование действительно будет избыточным и проще передать объект. *В таком случае возникает вопрос, а зачем было затаскивать навигацию, которая заведомо не может в объекты?* Если ответ - исторически так сложилось, то можно воспользоваться временной панацеей в виде упаковки в JSON строку и распаковку на принимающей стороне через маппер - тут, повторюсь, нужно взвешивать все плюсы и минусы конкретного случая и размера передаваемого объекта.

Что касается решения Гугла, то тут сложно сказать почему они так сделали. Решение действительно еще довольно сырое и в перспективе, скорее всего сильно изменится. Но важно также понимать, что и существующие решения по типу Cicerone тоже когда-то были сырыми =) Compose Navigation, лишь проба пера для нового подхода Compose. Все с чего-то начинается, а дальше посмотрим к чему придут.

Ответить

equeim 25 фев в 20:02

https://github.com/olshevski/compose-navigation-reimagined

Просто и минималистично, работает с любыми типами которые можно всунуть в Bundle.

Ответить

filatovvv 26 фев в 11:46

Отличное решение для того, чтобы на уровне Compose Navigation работать с объектами. Если для проекта не критично затаскивать библиотеку, то вариант подходящий (главное держать в уме, что любая библиотека - это всегда потенциальные риски и нужно исходить из всех плюсов минусов в конкретном случае).

Ответить

SpartakChampion 26 фев в 10:56

А как с диплинками обьстоят дела? Как из пуша попасть на детализацию токена?)

Ответить

filatovvv 26 фев в 12:34

На текущий момент никак, т.к. флоу не подразумевает инициализацию по диплинке) Но в перспективе, планирую завязаться на стандартный механизм работы с диплинками в Compose Navigation. Как реализую, обязательно добавлю в статью описание.

Ответить

Только полноправные пользователи могут оставлять комментарии. Войдите, пожалуйста.

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

Catx2 19 часов назад

Рога переходного периода (из ниоткуда в никуда)

26 мин 6.1K

Ретроспектива

+70 39 35 +35

dmitriyrudnev 22 часа назад

Методика D.I.Y. проекта. Стратегия выживания

Простой 9 мин 4.7K

Обзор

+59 42 13 +13

MaFrance351 23 часа назад

ПИУ-2. Оживляем плазменный текстовый индикатор из СССР

Средний 17 мин 4.2K

Туториал

+54 20 27 +27

dalerank 18 часов назад

Бобры-п[р]огромисты

Простой 4 мин 4.8K

Ретроспектива

+45 20 12 +12

CodeDroidX 18 часов назад

Анонимная P2P-база знаний, где никто не может подделать чужое авторство | Магия асимметричных ключей на практике

Средний 31 мин 3.7K

Туториал

+40 69 22 +22

romanova_ase 19 часов назад

Деплоим сайт в облако за рубль в месяц

8 мин 4.6K

Туториал

+25 73 23 +23

artur_speaking 23 часа назад

Как перестать переусложнять и начать жить

Простой 10 мин 9.2K

Из песочницы

+22 61 37 +37

AlbinaSem 22 часа назад

Найм хакеров глазами рекрутера: аналитика и советы для пентестеров и хедхантеров

14 мин 3K

Аналитика

+21 34 4 +4

Soro-Kin вчера в 12:18

Ключи Supermicro, SSM, VMWare и лекарство от жадности

5 мин 10K

+21 70 10 +10

Grigory_Otrepyev 23 часа назад

Как нанять работника в ИТ, если он реально нужен в 2024 году

Сложный 9 мин 7.5K

Мнение Recovery Mode

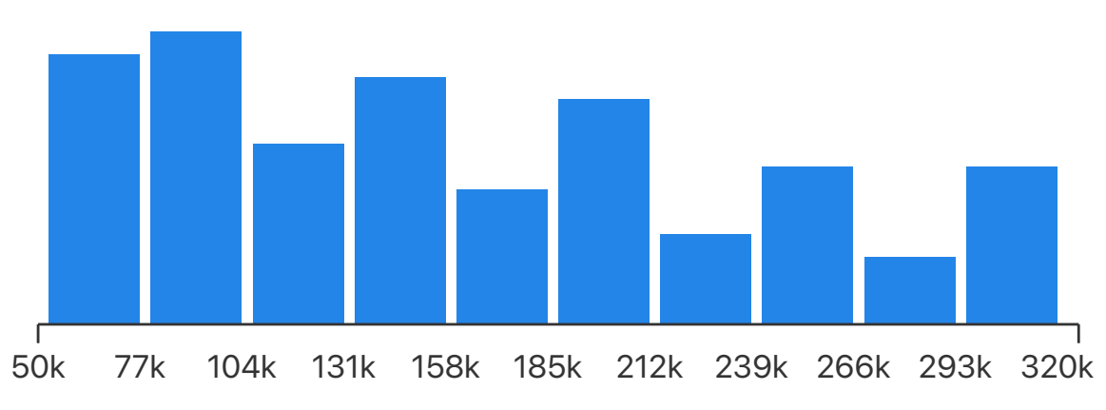
+19 24 8 +8

Показать еще

СРЕДНЯЯ ЗАРПЛАТА В IT

168 997 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 19 970 анкет, за 1-ое пол. 2024 года. Проверьте «в рынке» ли ваша зарплата или нет!



Проверить свою зарплату

ЧИТАЮТ СЕЙЧАС

Программированию конец?

17K 129 +129

Почему я бросил свою сеть по ремонту телефонов и причем здесь Онлифанс

48K 178 +178

Войти в IT. Как живётся молодым специалистам в государственных медучреждениях

2.6K 5 +5

«Яндекс» выпустил большое обновление Алисы и умных колонок

17K 18 +18

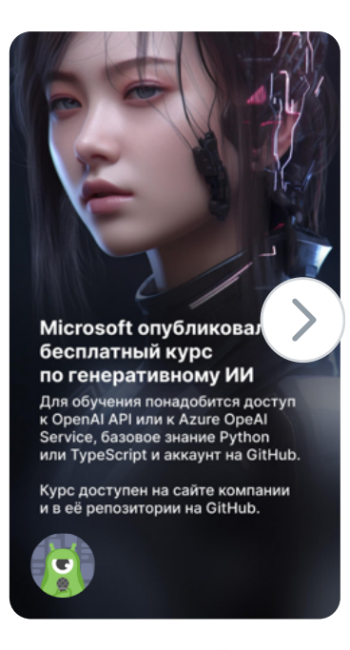
Microsoft прекратит поддержку подсистемы Windows Subsystem for Android (WSA) в Windows 11 с 5 марта 2025 года

2.4K 10 +10

ИСТОРИИ

HEROES of backend
Почта
Самооборона
Диск
Биллинг
Яндекс 360
Яндекс 360 нужны герои!
Выбери команду

Яндекс 360 призывает героев бэкенда


Microsoft опубликовала бесплатный курс по генеративному ИИ

Бесплатный ИИ-курс

РАБОТА

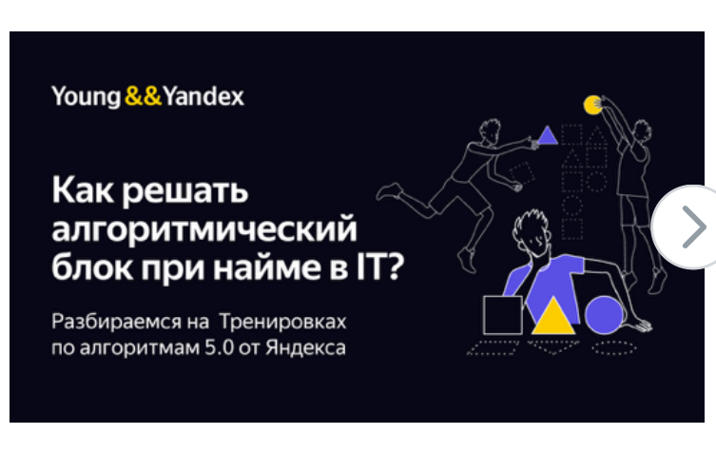
Android разработчик
32 вакансии

Swift разработчик
40 вакансий

iOS разработчик
30 вакансий

Все вакансии

БЛИЖАЙШИЕ СОБЫТИЯ


Young&&Yandex
Как решать алгоритмический блок при найме в IT?
Разбираемся на Тренировках по алгоритмам 5.0 от Яндекса

Серия занятий «Тренировки по алгоритмам 5.0» от Яндекса

1 марта – 19 апреля
19:00
Онлайн

Подробнее в календаре

Реклама

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам

© 2006–2024, Habr

Техническая поддержка Настройка языка